
AWS ECR – Elastic Container Registry

1. What is AWS ECR?

- A fully managed Docker container registry.
 - Securely stores, manages, and deploys Docker container images.
 - Integrated tightly with Amazon ECS, EKS, and Lambda.
 - Supports image scanning, lifecycle policies, and encryption.
 - Scales automatically to meet your container image storage needs.
-

2. Key Features

Feature	Description
Fully managed registry	No infrastructure to manage; AWS handles scaling.
Integration	Works natively with ECS, EKS, and AWS Lambda.
Image scanning	Finds vulnerabilities in container images.
Lifecycle policies	Automatically clean up old or unused images.
Encryption	At rest with AWS KMS and in transit.
Cross-region replication	Replicate images across AWS regions for redundancy and performance.

3. Core Concepts

Term	Description
Repository	Logical collection of container images.
Image	A Docker image, versioned by tags (e.g., latest, v1.0).
Image Tag	A label for an image version (e.g., latest).
Registry	Account-level container image storage (one per AWS account and region).

4. Creating a Repository in AWS ECR

Using AWS Console:

1. Open the **Amazon ECR** console.
 2. Click **Repositories** → **Create repository**.
 3. Enter a **Repository name**.
 4. Configure settings:
 - Visibility (Private by default).
 - Enable image scanning on push (optional).
 - Encryption (default KMS key or custom).
 5. Click **Create repository**.
-

Using AWS CLI:

```
aws ecr create-repository --repository-name my-app --region us-west-2
```

5. Pushing Docker Images to ECR

Step 1: Authenticate Docker to ECR

```
aws ecr get-login-password --region us-west-2 | docker login --username AWS --password-stdin <aws_account_id>.dkr.ecr.us-west-2.amazonaws.com
```

Step 2: Build your Docker image locally

```
docker build -t my-app .
```

Step 3: Tag your Docker image

```
docker tag my-app:latest <aws_account_id>.dkr.ecr.us-west-2.amazonaws.com/my-app:latest
```

Step 4: Push the image to ECR

```
docker push <aws_account_id>.dkr.ecr.us-west-2.amazonaws.com/my-app:latest
```

6. Pulling Images from ECR

Make sure your compute environment (ECS/EKS/EC2) has permission to pull.

Then run:

```
docker pull <aws_account_id>.dkr.ecr.us-west-2.amazonaws.com/my-app:latest
```

7. Managing Images and Repositories

- Use **lifecycle policies** to clean up old images automatically.
 - Enable **image scanning** to detect vulnerabilities.
 - Use **Cross-region replication** for disaster recovery and performance.
-

8. IAM Permissions

Ensure your IAM users or roles have these permissions to interact with ECR:

- `ecr:CreateRepository`
- `ecr:GetAuthorizationToken`
- `ecr:BatchCheckLayerAvailability`
- `ecr:PutImage`
- `ecr:InitiateLayerUpload`
- `ecr:UploadLayerPart`
- `ecr:CompleteLayerUpload`
- `ecr:BatchGetImage`
- `ecr:DescribeRepositories`
- `ecr:GetDownloadUrlForLayer`

9. Best Practices

Practice	Reason
Use fine-grained IAM policies	Limit access to specific repositories.
Enable image scanning	Detect vulnerabilities early.
Implement lifecycle policies	Reduce storage costs by cleaning unused images.
Use versioned tags	Avoid ambiguity and manage deployments.
Replicate critical repos	Improve availability across regions.
